

Medical-Report-Summarization (GitHub) – In-Depth Analysis

1. Code Functionality and Architecture

The **Medical Report Summarization System** is a web application for uploading medical reports (PDFs) and generating concise textual summaries. It is built with **Flask** (Python) for the web server and uses a familiar login/upload interface ¹. The frontend (HTML/CSS templates) provides pages for user login, report upload, and summary display. Key architectural components include:

- **Web Server (Flask):** Handles HTTP requests, user sessions, and serves HTML templates. All application logic (routes) is implemented in `app.py`, and the server can be run locally or deployed on a web host ² ¹.
- **Data Storage:** Patient and report metadata are stored in **MongoDB** (a NoSQL database) ¹ ³. The system also uses **ChromaDB** (a vector database) to store embeddings for any retrieval or RAG-style processing, as indicated by its ChromaDB and RAG tags ³.
- **PDF Processing:** Uploaded PDF reports are processed in two stages: first **text extraction** and then **summarization**. Text is extracted from PDFs using `pdfplumber` and LangChain's `PyPDFLoader` (wrapping libraries like PyMuPDF) ⁴. The extracted text is cleaned and concatenated for the next step.
- **Summarization (LLM + RAG):** The core summarization uses LangChain to run a pre-trained large language model. Specifically, it employs the **Mistral-7B-Instruct-v0.3** model (via HuggingFace) to generate structured medical summaries ⁵ ⁶. The extracted text is fed into a detailed prompt (template) that guides the model to output concise, organized summaries. This is effectively a retrieval-augmented generation (RAG) setup: the report text is converted into embeddings (ChromaDB) and the LLM crafts the summary.
- **PDF Generation:** Once a summary is generated, it is rendered to a PDF file so the user can download it. A helper script (`pdf_generator.py`) uses a Python PDF library (likely [FPDF](#)) to create the summary PDF ⁷. Users can view the summary on-screen and download the PDF with one click.

Overall, the system combines a traditional web MVC architecture (Flask + MongoDB) with modern NLP: PDF readers for extraction and a LLM (Mistral via LangChain) for summarization ⁸ ³. The data flow is as follows: user logs in and uploads a report → PDF text is extracted → text is summarized by the AI model → summary is saved and displayed/downloadable as PDF ⁹. This design supports **few-shot learning** (the prompt guides the model) and vector search (ChromaDB) in a streamlined web app.

2. Installation and Deployment

The repository provides both a direct (Python) setup and a Docker container option. To run the system:

- **Install Dependencies:** Clone the repo and install required Python packages (likely via `pip install -r requirements.txt`). Key libraries include Flask, PyMongo (or

`flask-pymongo`) for MongoDB, LangChain, `chromadb`, PDF processing (`pdfplumber`, `pypdf` / `PyMuPDF`), HuggingFace `transformers` (for Mistral-7B), and a PDF library (e.g. `FPDF`) for output.

- **Configure Database:** Ensure a MongoDB instance is running and accessible (the code likely uses a default `mongodb://localhost` connection unless configured otherwise).
- **Run the Flask App:** Launch the server with `python app.py` or `flask run`. The README notes the Flask server “runs the application locally or on a web server”². By default it will listen on a local port (e.g. 5000). Navigate to the app’s URL to access the UI.
- **Docker Deployment (optional):** A **Dockerfile** is included for containerized deployment. One can build the image (e.g. `docker build -t med-summarizer .`) and run it (`docker run -p 5000:5000 med-summarizer`). The container will install dependencies and start the Flask app automatically. This simplifies running all components (Flask, model, etc.) together.

No special API keys or external services are explicitly required (the Mistral model can be fetched from HuggingFace). In summary, setup is the standard “pip install + configure DB + run Flask” workflow, or simply “build and run the Docker image.” The system is fully self-contained in the codebase.

3. Dependencies and Requirements

This project targets **Python 3.x** and relies on several major libraries and services:

- **Flask** – Web framework for the server and routes.
- **MongoDB** – NoSQL database for patient and report data (used via a Python MongoDB driver).
- **LangChain** – Framework used to orchestrate the LLM summarization pipeline.
- **Chromadb** – Vector database for storing embeddings (suggested by project topics).
- **HuggingFace Transformers** – To load the **Mistral-7B-Instruct-v0.3** model for summary generation⁵ ⁶.
- **PDF Processing:** Libraries such as `pdfplumber` (for OCR-free text extraction from PDFs) and `PyPDFLoader` (LangChain loader)⁴.
- **PDF Generation:** A PDF library (likely **FPDF**) to create summary PDFs⁷.
- **Other Utilities:** Common packages like `Flask-CORS`, `requests`, etc., and Python’s built-in `logging` module¹⁰.

System requirements include enough RAM/CPU to run a 7B-parameter LLM (ideally with GPU, although Mistral-7B-Instruct may be run on CPU for small loads). The Dockerfile (not shown) probably installs all dependencies automatically. In short, a modern Linux/Windows system with Python 3, MongoDB installed, and Internet access (for huggingface model download) is needed to run this project¹ ³.

4. Performance / Benchmarks

The repository does **not** report any performance benchmarks or evaluation metrics (e.g. ROUGE scores) for the summaries. It appears to be a proof-of-concept/demo rather than a research benchmark. The focus is on functional operation (upload → summarize) rather than quantitative evaluation. No training or test datasets or accuracy measurements are provided. Users would have to gauge output quality qualitatively.

5. Comparison with Similar Projects

Several other GitHub projects aim at medical report summarization or related NLP tasks. Below is a comparison in terms of design, usability, and popularity:

Repository	Approach / Architecture	Interface	GitHub Stars / Forks
This Project (Dhanush-R-git/Medical-Report-Summarization)	Flask-based web app using LangChain + HuggingFace LLM (Mistral-7B-Instruct) with MongoDB and Chroma vector DB ¹ ⁵ . PDF text is extracted via pdfplumber / PyPDFLoader and summarized in-application ⁴ ⁵ .	Web UI (HTML/CSS)	1 star, 0 forks ¹¹
Arinjay-04/Medical-Report-Summarization-Generalization	Standalone Python script. Uses pdfplumber for PDF text extraction and spaCy for NLP, then calls the OpenAI API for summarization ¹² . Focuses on keyword-guided extractive summarization. ¹³	CLI only (no UI)	0 stars, 0 forks ¹⁴
PrinceKhunt16/Healthcare-AI-Langchain-FAISS-DB-Ollama-Grok...	Full-stack web application (React frontend + Flask backend) integrating LangChain with a local LLM (via Ollama/Grok) and FAISS for retrieval ¹⁵ ¹⁶ . Includes many features (chatbot, report explanation, appointments).	Web UI (React + Flask)	0 stars, 1 fork ¹⁷
databricks-industry-solutions/jsl-medical-text-summarization	Spark NLP pipeline (John Snow Labs healthcare models). It provides Python/Spark notebooks using specialized clinical summarization models (e.g. JSL's finetuned transformers) ¹⁸ ¹⁹ . No web UI; used as a library/API.	Notebook/ API (Spark)	6 stars, 5 forks ¹⁹

- **Architecture:** Our project uses a modern LLM + RAG approach (LangChain + Chroma + Mistral) in a lightweight Flask app. By contrast, Arinjay-04 is much simpler: it extracts sentences via spaCy and then uses OpenAI (GPT) only to “explain” the text ¹² . PrinceKhunt16’s hackathon app is architecturally similar (LangChain + vector DB) but more complex, with a React front-end and multiple features ¹⁵ . The Databricks project is entirely different: it uses Apache Spark and specialized NLP libraries (Spark NLP Healthcare models) in batch mode ¹⁸ , not a user-facing web service.
- **Usability:** Our system offers a user-friendly web interface for uploading reports and viewing summaries. Arinjay-04 has no UI (it’s a script you run on the command line) and requires manual keyword input. PrinceKhunt16 has a polished (hackathon) multi-page app with React, while the Databricks solution is meant for developers/data scientists (no GUI). In terms of ease of use, our project sits between: simpler to run than the Spark pipeline, but more involved than Arinjay’s one-shot script.

- **Popularity:** All these repos are modestly popular. As of this writing, our repo has **1 star** ¹¹. Arinjay-04 has **0 stars** ¹⁴, and PrinceKhunt16 has **0 stars and 1 fork** ¹⁷. The Databricks JSL repo has garnered **6 stars and 5 forks** ¹⁹ (reflecting corporate backing). Star counts suggest that none of these are widely used/popular projects, but the Databricks and John Snow Labs solution is the best-known by reputation.

Overall, our project's architecture (Flask + LangChain + Chroma) is comparable to PrinceKhunt16's but simpler and more focused. Unlike Arinjay-04, we leverage an open-source LLM (Mistral) instead of a paid API. Compared to the Databricks/Spark approach, our code is lightweight (Python server vs big data) and easier to deploy, though we sacrifice large-scale performance.

6. Author Background and Activity

The repository owner **Dhanush R** (GitHub user *Dhanush-R-git*) is a developer from Chennai, India (affiliated with RMK Engineering College ²⁰). His GitHub profile indicates interests in data analytics, AI, and large language models ²¹. He has **17 public repositories** and about **8 stars total** on his account ²², with many recent projects in healthcare and machine learning (e.g. mental health analysis, disease diagnosis, etc.) updated as recently as early 2025 ²³ ²⁴. The Medical-Report-Summarization repo is **pinned** on his profile ²⁵, suggesting it is one of his showcase projects. The commit history for this repo shows it was initially created and populated in August 2024 (9 commits on one day) ²⁶, and GitHub notes it was updated in April 2025 ²³. His activity level appears steady (multiple ML/AI projects with updates in 2024-2025), though the Medical-Report-Summarization repo itself had most development early on and only minor updates since.

In summary, Dhanush is an emerging AI/ML practitioner focusing on healthcare applications. The Medical-Report-Summarization project seems to be part of his portfolio work in summarization and RAG, and he maintains an active GitHub profile with related projects ²² ²¹.

Sources: Repository README and code, GitHub profile and related project pages ¹ ⁵ ⁹ ¹² ¹⁵ ¹⁹ ¹¹ ²² ²¹.

1 2 3 4 5 6 7 8 9 10 11 GitHub - Dhanush-R-git/Medical-Report-Summarization: The Medical Report Summarization System is a web-based application designed to facilitate the management and summarization of medical reports.

<https://github.com/Dhanush-R-git/Medical-Report-Summarization>

12 13 14 GitHub - Arinjay-04/Medical-Report-Summarization-Generalization-

<https://github.com/Arinjay-04/Medical-Report-Summarization-Generalization->

15 16 17 GitHub - PrinceKhunt16/Healthcare-AI-Langchain-FAISS-Database-Olama-Grok-React.js-Flask

<https://github.com/PrinceKhunt16/Healthcare-AI-Langchain-FAISS-Database-Olama-Grok-React.js-Flask>

18 19 GitHub - databricks-industry-solutions/jsl-medical-text-summarization

<https://github.com/databricks-industry-solutions/jsl-medical-text-summarization>

20 21 22 24 25 Dhanush-R-git (Dhanush R) · GitHub

<https://github.com/Dhanush-R-git>

23 Dhanush-R-git (Dhanush-R-git) / Repositories · GitHub

<https://github.com/Dhanush-R-git?tab=repositories>

26 Commits · Dhanush-R-git/Medical-Report-Summarization · GitHub

<https://github.com/Dhanush-R-git/Medical-Report-Summarization/commits/main/>